Algorytmy ewolucyjne

Zasada działania, obszary zastosowań, biblioteki programistyczne, zastosowanie do wybranego problemu optymalizacyjnego oraz eksperymenty

Wiktor Skrzypnik, Jakub Zając

WSZiB, 2026

1. Zasada działania i zastosowania

Algorytmy ewolucyjne to stochastyczne metody przeszukiwania i optymalizacji, których działanie jest inspirowane naturalnymi procesami ewolucyjnymi, takimi jak dobór naturalny, krzyżowanie oraz mutacja genetyczna [1]. Stanowią one istotną gałąź sztucznej inteligencji i należą do klasy algorytmów metaheurystycznych. Pozwala im to skutecznie radzić sobie z trudnymi problemami optymalizacyjnymi, w których przestrzenie rozwiązań są ogromne, nieliniowe lub niedostępne dla tradycyjnych metod analitycznych [2].

Zasada działania algorytmu ewolucyjnego

Schemat działania algorytmu ewolucyjnego opiera się na iteracyjnym przetwarzaniu populacji potencjalnych rozwiązań (osobników). Każdy osobnik reprezentuje zakodowany w określony sposób punkt w przestrzeni poszukiwań (tzw. chromosom lub genotyp). Proces ten przebiega w następujących krokach:

1. **Inicjalizacja populacji:** Generowana jest początkowa populacja osobników (najczęściej losowo w obrębie dopuszczalnej przestrzeni rozwiązań).
2. **Ocena przystosowania** (Fitness Evaluation): Dla każdego osobnika obliczana jest wartość funkcji przystosowania (fitness function), która określa stopień dopasowania danego rozwiązania do stawianego kryterium optymalizacyjnego.
3. **Selekcja** (Selection): Wybierane są osobniki wyróżniające się najlepszymi wynikami. Zyskują one szansę na „przeżycie” i przekazanie swoich cech kolejnemu pokoleniu zgodnie z zasadą selekcji naturalnej (np. metodą ruletkową lub turniejową) [1].
4. **Operatory genetyczne:**
    - Krzyżowanie (Crossover): Łączenie materiału genetycznego wybranych osobników rodzicielskich w celu wygenerowania potomstwa dziedziczącego cechy obu stron.
    - Mutacja (Mutation): Losowa, niewielka modyfikacja wybranych elementów genotypu potomstwa, co pozwala na wprowadzenie nowej różnorodności i ucieczkę z minimów lokalnych [2].
5. **Generacja nowej populacji i warunek stopu:** Procesy selekcji, krzyżowania i mutacji prowadzą do utworzenia nowej generacji. Algorytm powtarza te iteracje do momentu spełnienia warunku stopu (np. osiągnięcia satysfakcjonującej wartości funkcji celu lub wyczerpania limitu pokoleń).

Wybrane obszary zastosowań

Dzięki uniwersalności i elastyczności algorytmy ewolucyjne znajdują szerokie zastosowanie w nauce i przemyśle:

- Logistyka i transport: Rozwiązywanie problemów kombinatorycznych, takich jak Problem Komiwojażera (TSP) czy optymalizacja tras pojazdów dostawczych (VRP) [2].
- Inżynieria i projektowanie: Automatyczne generowanie optymalnych kształtów konstrukcji, optymalizacja topologii komponentów pod kątem wytrzymałości i minimalizacji masy [1].
- Sterowanie i robotyka: Optymalizowanie trajektorii ruchu robotów oraz sterowanie systemami dynamicznymi [2].
- Uczenie maszynowe: Dobór optymalnych hiperparametrów modeli oraz automatyczne dostrajanie struktur systemów inteligentnych [1].

2. Biblioteki programistyczne

DEAP

1. **Instalacja:** pip install deap.
2. **Możliwości:** Tworzenie populacji, definiowanie funkcji przystosowania oraz prowadzenie kolejnych pokoleń algorytmu. [3], [4].
3. **Dostępne algorytmy:** Algorytmy genetyczne, strategie ewolucyjne, programowanie genetyczne i algorytmy wielokryterialne. [3].
4. **Zalety i wady:** Zaletami są elastyczność oraz gotowe operatory selekcji, krzyżowania i mutacji. Wadą jest konieczność samodzielnej konfiguracji reprezentacji rozwiązania i funkcji celu.

Jenetics

1. **Instalacja:** Bibliotekę dodaje się do projektu Java za pomocą Maven, umieszczając zależność io.jenetics:jenetics w pliku pom.xml. [5].
2. **Możliwości:** Definiowanie genotypów, populacji, funkcji przystosowania i obserwowanie wyników kolejnych generacji. [5], [6].
3. **Dostępne algorytmy:** Algorytmy genetyczne oraz operatory selekcji, mutacji i krzyżowania. [5].
4. **Zalety i wady:** Zaletami są przejrzysta struktura i możliwość wykorzystania wielowątkowości. Wadą jest konieczność znajomości Javy oraz bardziej rozbudowana konfiguracja.

NetworkX

1. **Instalacja:** pip install networkx.
2. **Możliwości:** Tworzenie i analiza grafów. W projekcie wierzchołki oznaczają punkty lub miasta, a krawędzie drogi z informacją o długości. [7].
3. **Dostępne algorytmy:** Wyszukiwanie najkrótszych ścieżek oraz analiza połączeń w grafie. [7].
4. **Zalety i wady:** Zaletą jest prosta obsługa grafów i atrybutów krawędzi. Wadą może być mniejsza wydajność przy bardzo dużych grafach.

Matplotlib i Tkinter

1. **Instalacja:** pip install matplotlib. Tkinter jest zwykle częścią standardowej instalacji Pythona.
2. **Możliwości:** Matplotlib tworzy wykresy i wizualizuje trasę, a Tkinter umożliwia stworzenie prostego interfejsu graficznego. [8], [9].
3. **Dostępne algorytmy:** Biblioteki te nie zawierają algorytmów ewolucyjnych; służą do prezentacji wyników i obsługi interfejsu.
4. **Zalety i wady:** Zaletą jest prosta obsługa i szybka wizualizacja wyników. Wadą Tkintera jest ograniczona funkcjonalność w porównaniu z bardziej rozbudowanymi frameworkami.

3. Zaimplementowany algorytm rozwiązujący wybrany problem optymalizacyjny

[opis wybranego problemu optymalizacyjnego (np. optymalizacja funkcji, komiwojażer, generowanie optymalnych konstrukcji (np. http://www.boxcar2d.com/), optymalizacja wielokryterialna, wielokryterialna optymalizacja trasy drona/statku z uwzględnieniem przeszkód/pogody) oraz szczegółowy opis algorytmu ewolucyjnego użytego do jego rozwiązania: pseudokod wraz z opisem słownym, sposób reprezentacji rozwiązania w genotypie osobnika, wartości parametrów (liczebność populacji, prawdopodobieństwa mutacji i rekombinacji oraz inne w zależności od użytego algorytmu) wraz z komentarzami]

4. Wyniki eksperymentów

[wartości parametrów algorytmu ustawiane dla poszczególnych eksperymentów, jeżeli różnią się od tych podanych w sekcji 3, skomentowane wykresy pokazujące średnie wyniki z odchyleniami standardowymi z co najmniej 10 uruchomień algorytmu (wykresy powinny pokazywać: najlepszy, najgorszy i średni fitness w populacji w trakcie trwania eksperymentu), omówienie najlepszego znalezionego rozwiązania.]

5. Podsumowanie

[podsumowanie rezultatów projektu oraz uzyskanych wyników eksperymentów]

Bibliografia

1. Goldberg, D. E. (1989). Genetic Algorithms in Search, Optimization, and Machine Learning. Addison-Wesley.
2. Bodenhofer, U. (2003). Genetic Algorithms: Theory and Applications. Lecture Notes, Johannes Kepler University Linz.
3. Fortin, F.-A. i in. (2012). DEAP: Evolutionary Algorithms Made Easy. Journal of Machine Learning Research, 13, 2171–2175.
4. DEAP Documentation. DEAP User Guide. https://deap.readthedocs.io/
5. Jenetics Documentation. Jenetics: Genetic Algorithm, Evolutionary Algorithm and Genetic Programming Library. https://jenetics.io/
6. Jenetics API Documentation. https://jenetics.io/javadoc/8.2.0/
7. Hagberg, A., Swart, P., Schult, D. (2008). Exploring Network Structure, Dynamics, and Function using NetworkX. Proceedings of the 7th Python in Science Conference.
8. Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering, 9(3), 90–95.
9. Python Software Foundation. Tkinter — Python interface to Tcl/Tk. https://docs.python.org/3/library/tkinter.html